Intern Coding Challenge: Personal Finance Tracker System

Objective:

Design and implement a Personal Finance Tracker to record expenses and income, auto-categorize transactions, and track financial health. Emphasize data accuracy, smart classification, and clean API design.

Your solution should be fully functional, adhere to OpenAPI standards, and demonstrate best practices in backend, frontend, database, and SDK development.

Tasks & Requirements

1. Backend Development (FastAPI & SQLite)

Implement a FastAPI service with the following endpoints:

- **POST /transactions/** → Add a transaction (amount, description, type).
- **GET /transactions/** → Get all transactions.
- **GET /transactions/summary** → Monthly income/expense summary per category.
- **DELETE /transactions/{transaction_id}** → Delete a transaction.
- **PATCH /transactions/{transaction_id}/reclassify** → Manually correct auto-categorization.

Use SQLite as the database.

Ensure the API follows OpenAPI standards (Swagger UI should be accessible).

Implement error handling (appropriate 4xx/5xx).

Use dependency injection for database operations.

Write unit tests for the backend.

🚨 Trick Logic in Backend (Hidden Complexity)

- Auto-categorize transactions by description (e.g., 'Uber' → Transport).
- Detect duplicate transactions (same amount + description + time window).
- Validate transaction type (`income` or `expense`).
- Aggregations group by month and category; expenses stored as negative but displayed as absolute values.

Your implementation should correctly enforce the rules above.

📌 Examples:
- ✅ 'McDonald's' auto-categorized as 'Food'
- ❌ 'Rent' entered as 'income' → validation error
- ✅ Reclassify 'Uber' from 'Transport' to 'Travel' → update overrides auto-tag

---

2. Database (SQLite)

Create the following database tables:

```
CREATE TABLE transactions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    description TEXT NOT NULL,
    amount REAL NOT NULL,
    transaction_type TEXT CHECK (transaction_type IN
('income','expense')) NOT NULL,
    category TEXT,
    auto_tagged BOOLEAN NOT NULL DEFAULT 1,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Use Alembic for database migrations.

Provide a sample SQL file (`seed_data.sql`) to populate initial test data.

---

3. Frontend Client (ReactJS)

Develop a ReactJS-based frontend.

Use Axios for making API calls.

The frontend should allow users to:
- Add a transaction (amount, description, type)
- List with filters (month/type/category)
- View monthly summaries (grouped by category)
- Reclassify a transaction category
- (Bonus) Pie chart of monthly expenses

Ensure the frontend interacts only via API calls (no direct database access).

Style is not a priority, but functionality must work correctly.

📌 Bonus: Real-time updates where meaningful.

---

4. Platform SDK (Generated via OpenAPI Generator CLI)

Instead of manually writing an SDK, generate a Python SDK using the OpenAPI Generator CLI. This SDK should allow interaction with the API.

Instructions to Generate SDK

```bash
npm install -g @openapitools/openapi-generator-cli
```

```
openapi-generator-cli generate -i http://localhost:8000/openapi.json -g
python -o finance_sdk
```

Expected SDK Usage (After Generation & Testing):

```python
from finance_sdk.api.transactions_api import TransactionsApi
from finance_sdk import ApiClient

client = ApiClient()
api = TransactionsApi(client)

# Example usage
# result = api.<your_method_here>()
# print(result)
```

---

5. Development Setup Script (`setupdev.bat`)

Create a Windows batch script to set up the environment:

```bat
@echo off
echo Setting up backend...
python -m venv env
call env\Scripts\activate
pip install -r requirements.txt
alembic upgrade head

echo Setting up frontend...
cd frontend
npm install
cd ..

echo Setup complete.
```

---

6. Execution Script (`runapplication.bat`)

Create a Windows batch script to start the full application.

```bat
@echo off
echo Starting backend...
call env\Scripts\activate
python main.py

echo Starting frontend...
cd frontend
npm start
```

Evaluation Criteria

✅ Code Quality: Well-structured, clean, and modular code.
✅ Correct API Implementation: API should work correctly and follow OpenAPI standards.
✅ Proper Error Handling: Handles bad inputs and errors gracefully.
✅ Platform SDK Generation: SDK must be generated correctly using OpenAPI Generator CLI.
✅ Frontend Integration: The ReactJS frontend should correctly communicate with the backend.
✅ Automation Scripts: setupdev.bat and runapplication.bat should work as expected.
✅ Unit Tests: Backend should include test cases.
✅ Documentation: Include a README with installation and execution steps.
✅ Backend Trick Logic: Ensure candidates correctly implement the specified rules.

Deliverables

- Backend (FastAPI) source code
- SQLite database & migration files
- ReactJS frontend code
- Python SDK (generated via OpenAPI Generator CLI)
- Setup & Execution Scripts (`setupdev.bat`, `runapplication.bat`)
- Unit tests for backend
- README explaining how to set up and run the project

⏳ **Time Limit:** 2-3 Days

💡 Additional Notes

- You are **NOT** allowed to use Docker.
- Ensure proper API validation and error handling.
- The frontend does **not** need to be fancy – focus on functionality.
- Bonus points for implementing extra features like charts or real-time updates.